findings

Semgrep `go.lang.security.audit.dangerous-exec-command` — Taint Triage

Field	Value
Date	2026-06-22
Rule	`go.lang.security.audit.dangerous-exec-command` (audit-class, input-provenance)
Scope	4 findings under `helix_code/internal/llm/`
Method	§11.4.102 systematic-debugging taint analysis (codegraph + grep), READ-ONLY
Evidence	Provenance traced to source for every site; captured grep/Read excerpts below
Constitutional context	helix_code DELIBERATELY uses `os/exec` (anti-BLUFF-003: `handleCommand` MUST really execute). Static/trusted exec is EXPECTED and CORRECT. The only question: does UNTRUSTED input reach the command NAME or get concatenated into a SHELL STRING?

Verdict summary

#	file:line	function	arg-pr
1	`internal/llm/auto_llm_manager.go:520`	`AutoLLMManager.autoBuildProvider`	`exec.` `scrip` STATI `provi` `local` literal

#	file:line	function	arg-pr
			set fro into Bu
2	`internal/llm/local_llm_manager.go:439`	`LocalLLMManager.buildProvider`	`exec. c",pr` static (copie line 26 input.
3	`internal/llm/model_converter.go:303`	`ModelConverter.runConversion`	`exec. tool. tool.` (hardc `initi` 488-54 `confi` SEPAR `build` (`--fo` `optim` `local` but as NO sh
4	`internal/llm/model_download_manager.go:479`	`ModelDownloadManager.convertModel`	`exec. args. "pyth initi` 715-73 `{inpu` replac array.

Provenance detail (captured evidence)

Findings 1 & 2 — `BuildScript` is static, never user-controlled

- `auto_llm_manager.go:396` `for name, providerDef := range providerDefinitions` → `BuildScript: providerDef.BuildScript` (line 405).
- `local_llm_manager.go:252` `for name, definition := range providerDefinitions` → `BuildScript: definition.BuildScript` (line 260).

- `providerDefinitions` is `var providerDefinitions = map[string]*LocalLLMProvider{...}` (`local_llm_manager.go:51`) — a compile-time map literal. Every `BuildScript` is a hardcoded constant, e.g. `"python3 -m pip install -e ."`, `"make build"`, `"pip install -r requirements.txt"`, `"mkdir -p build && cd build && cmake .. && make -j$(nproc)"`.
- grep for any assignment INTO `BuildScript` (`BuildScript =`/`BuildScript:`/Unmarshal/viper) returns ONLY: the static literals (lines 59-201), the two field copies from the static map (405, 260). NO deserialization path, NO HTTP/CLI/config-file setter targets this field. `loadConfiguration()` (auto_llm_manager.go:361) is a no-op stub that does not parse YAML into providers.
- `bash -c <script>` IS shell interpretation, so these are the two true "shell-string" sites — but the string is a trusted constant, so injection is impossible by construction.

Findings 3 & 4 — static command name + arg-array (no shell)

- `model_converter.go:303` `exec.CommandContext(ctx, tool.Command, job.Args...)`. `tool.Command` from `initializeAllConversionTools()` (line 482) — all entries `Command: "python"`.
- `job.Args = c.buildCommand(tool, config)` (line 147). `buildCommand` (line 364): `copy(args, tool.Args)` then `append(args, "--input", config.SourcePath)`, `append(args, "--quantize", config.Quantization.Method)`, etc. — each user value is a DISTINCT slice element.
- User reachability CONFIRMED: `cmd/local_llm.go:799 runConvertModel` → `ConvertModel(ctx, config)` populates `config` from cobra flags `--format`/`--quantize`/`--optimize`/`--hardware`. This is genuine user input.
- WHY STILL SAFE: `exec.Command`/`CommandContext` exec the binary directly via `execve` — they do NOT spawn a shell. argv elements are passed verbatim; a value like `; rm -rf /` becomes one literal argument to `python`, not a new command. The command NAME (`tool.Command`) is never user-derived. This is precisely the arg-array (not shell-string) mitigation the rule recommends. Note: `$INPUT`/`$OUTPUT` literals in two tool arg templates are NOT shell-expanded by exec — they are passed verbatim and handled by the python tool itself.
- `model_download_manager.go:479` `exec.Command(tool.Command, args...)`: `tool.Command` static `"python"` (`initializeConversionTools` line 715), `tool.Args` static with `{input}`/`{output}` placeholders (`strings.ReplaceAll`, line 467-469) replaced by file paths, passed as argv array. Same safe pattern.

Conclusion

- **SAFE: 4 · REAL-RISK: 0 · UNCONFIRMED: 0**
- No real injection risk at any of the 4 sites. Findings 1 & 2 use `bash -c` but only with hardcoded constants from the static `providerDefinitions` map. Findings 3 & 4 use the arg-array pattern with a static command name; even genuine CLI-flag user input enters as discrete argv elements with no shell, so command injection is impossible.
- These are audit-class true-positives-by-pattern but false-positives-by-provenance. All 4 are candidates for `// nosemgrep: go.lang.security.audit.dangerous-exec-command <reason>` suppression (NOT applied here — READ-ONLY task).
- HARDENING NOTE (defense-in-depth, optional, NOT a current vuln): if `providerDefinitions` is ever made user/config-loadable in future, findings 1 & 2 would flip to REAL-RISK because of the `bash -c` shell-string form. A guard test asserting `BuildScript` is never deserialized would lock this invariant.